



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO

DISCIPLINA: LABORATÓRIO DE ALGORITMOS E ESTRUTURAS DE DADOS II

DOCENTE: KENNEDY REURISON LOPES

DISCENTES:

ANABEL MAFALDO ALVES

GIOVANI GABRIEL SALDANHA PEREIRA

SAMUEL SOUSA DE OLIVEIRA

**RELATÓRIO EXPERIMENTAL: SISTEMA DE VERIFICAÇÃO DE CADASTRO
COM TABELA HASH E FILTRO DE BLOOM**

1. Introdução

Este relatório documenta o desenvolvimento de um sistema de verificação de cadastro de usuários que combina duas estruturas de dados complementares: a Tabela Hash, responsável pelo armazenamento e busca exata dos registros, e o Filtro de Bloom, utilizado como pré-filtro probabilístico para evitar consultas desnecessárias à estrutura principal. O sistema descarta imediatamente elementos definitivamente ausentes, reduzindo o custo total das buscas. A interface do sistema é via terminal, com menu interativo para operações individuais e execução de experimentos em lote.

2. Implementação

Tabela Hash

A Tabela Hash foi implementada com encadeamento externo (listas ligadas) e função hash FNV-1a (Fowler–Noll–Vo). O tamanho foi fixado em 12.007 posições, um número primo escolhido deliberadamente para reduzir agrupamentos e distribuir melhor os índices calculados pelo operador módulo. Com esse dimensionamento, o fator de carga esperado é de aproximadamente 0,08 para 1.000 elementos e 0,83 para 10.000 — valores que mantêm a busca em $O(1)$ médio. O encadeamento externo foi preferido ao endereçamento aberto por não degradar com fatores de carga próximos de 1.

Filtro de Bloom

O Filtro de Bloom foi implementado com um vetor de bits e múltiplas funções hash. Para o menu interativo, o vetor foi fixado em 65.536 bits — adequado para uso manual com poucos elementos. Para os experimentos automáticos, o dimensionamento é calculado dinamicamente por cenário usando as fórmulas ótimas com taxa de falsos positivos $p = 1\%$: $m = -(n \times \ln p) / (\ln 2)^2 \rightarrow k = (m / n) \times \ln 2$ Para 100.000 registros: $m \approx 958.506$ bits (~117 KB) e $k = 7$ funções hash — muito menor que o espaço necessário para armazenar as chaves diretamente. As funções hash utilizadas são DJB2, SDBM e FNV-1a, combinadas com variações por seed para simular k funções independentes sem implementá-las separadamente.

3. Experimentos e Resultados

Para cada cenário, foram geradas consultas com nomes aleatórios não cadastrados para medir falsos positivos. O tempo foi medido com `clock()` sobre o conjunto total de buscas. Os arquivos de entrada contém identificadores no formato 8 letras + 3 dígitos, gerados aleatoriamente pelo gerador incluso no projeto

Quantidade	Tempo sem Bloom	Tempo com Bloom	Falsos Pos. (%)	Bits usados	Funções hash
1000	0,001000	0,000000	1,60%	9.586	7
10.000	0,004000	0,006000	0,97%	95.851	7
100.000	0,172000	0,219000	0,95%	958.506	7

Tabela 1 — Resultados experimentais comparativos.

4. Análise

Tempo sem Bloom

O tempo de busca direta na Tabela Hash cresce proporcionalmente ao volume: de 0,001s para 1.000 registros até 0,172s para 100.000. Esse crescimento é esperado, pois a busca percorre mais entradas e enfrenta mais colisões com o aumento do fator de carga.

Tempo com Bloom

No cenário de 1.000 registros, o tempo com Bloom (0,000s) foi menor que sem Bloom (0,001s), demonstrando que o filtro evitou acessos à hash de forma eficiente. Nos cenários maiores (10.000 e 100.000), o tempo com Bloom foi ligeiramente superior — 0,006s contra 0,004s, e 0,219s contra 0,172s, respectivamente. Isso ocorre porque, como todos os elementos consultados estavam cadastrados, o Bloom sempre disse "possivelmente existe", adicionando o custo das 7 funções hash sem eliminar nenhuma consulta à Tabela Hash. O ganho real do Bloom ocorre quando há consultas negativas (elementos não cadastrados), situação em que ele evita completamente o acesso à hash.

Falsos Positivos

A taxa de falsos positivos manteve-se próxima de 1% em todos os cenários, confirmando o dimensionamento teórico. O melhor resultado foi no cenário de 100.000

registros (0,95%), e o pior em 1.000 registros (1,60%), o que é esperado: com poucos elementos e um vetor de 9.586 bits, pequenas variações estatísticas impactam mais a taxa.

5. Respostas às perguntas do experimento

O Filtro de Bloom reduziu consultas na Tabela Hash? Sim. O filtro funciona como camada de pré-filtro: ao retornar "definitivamente não existe", evita qualquer acesso à Tabela Hash. Apenas resultados positivos (reais ou falsos) exigem checagem na estrutura principal. O benefício é mais evidente em cargas de trabalho com alta proporção de buscas negativas.

Como o tamanho do vetor impactou os resultados? O tamanho do vetor é determinante para a precisão. No menu interativo, o vetor fixo de 65.536 bits seria insuficiente para 100.000 elementos — resultaria em taxa de falso positivo próxima de 100%, pois quase todos os bits ficariam setados. Por isso, nos experimentos automáticos, o vetor é dimensionado dinamicamente por cenário usando a fórmula ótima, garantindo ~1% de falso positivo independentemente do volume.

O número de funções hash alterou a precisão? Sim. Poucas funções aumentam a taxa de falsos positivos por não espalharem bem os bits. Funções em excesso aumentam o custo computacional por consulta. O valor $k = 7$, calculado pela fórmula ótima, representa o ponto de equilíbrio entre precisão e desempenho para os três cenários.

Em qual cenário o Bloom foi mais vantajoso? No cenário de 1.000 registros, onde o tempo com Bloom (0,000s) foi inferior ao sem Bloom (0,001s). De forma geral, o Bloom é mais vantajoso quanto maior for a proporção de buscas negativas no sistema, pois é exatamente nesses casos que ele elimina o acesso à Tabela Hash.

6. Conclusão

A combinação Tabela Hash + Filtro de Bloom mostrou-se eficaz para verificação de cadastro em larga escala. O Filtro de Bloom eliminou consultas desnecessárias com custo constante e taxa de falsos positivos controlada (~1%), confirmando as previsões teóricas. Todas as estruturas foram implementadas do zero em C, com dimensionamento baseado em

análise quantitativa das fórmulas ótimas. Os resultados experimentais demonstraram que a escolha correta do tamanho do vetor de bits e do número de funções hash é essencial para obter o equilíbrio entre precisão, desempenho e consumo de memória.

7. Como obter, compilar e executar o programa

O código-fonte está disponível no repositório público do GitHub: <https://github.com/GioOGabriel/ED-II>.

Para clonar o repositório: `git clone https://github.com/GioOGabriel/ED-II.git`

Com o GCC instalado, a compilação é feita a partir da raiz do projeto com o comando: `gcc src/main.c src/hash.c src/bloom.c -o programa -lm` A flag `-lm` é obrigatória pois o Filtro de Bloom utiliza funções da biblioteca matemática (`log`, `pow`, `exp`, `ceil`).

No Windows com MinGW, substitua: `-o programa` por `-o programa.exe`. A execução deve ser feita a partir da raiz do projeto para que os arquivos de dados em `data/` sejam encontrados corretamente: `./programa`

8. Referências

BLOOM, B. H. Space/time trade-offs in hash coding with allowable errors. **Communications of the ACM**, v. 13, n. 7, p. 422-426, jul. 1970.

CORMEN, T. H. et al. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

KIRSCH, A.; MITZENMACHER, M. Less hashing, same performance: building a better bloom filter. **Random Structures & Algorithms**, v. 33, n. 2, p. 187-218, 2008.

KNUTH, D. E. **The Art of Computer Programming, Vol. 3: Sorting and Searching**. 2. ed. Reading: Addison-Wesley, 1998.